

Kria Yocto Vitis Vision Library App

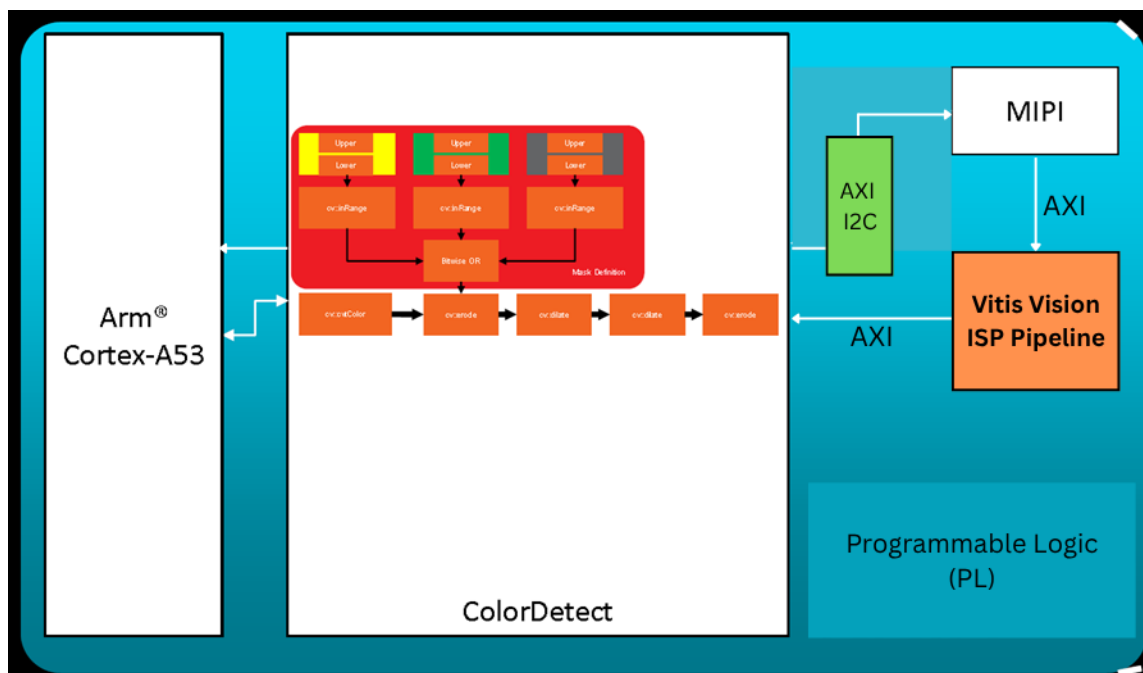
Getting Started Guide

A. Overview

This kria app utilizes the Vitis Vision Library ColorDetect HLS kernel based on the MIPI data received from Raspberry Pi Camera v2.

This system is running at Kria Yocto tool and in the background it uses 2025.2 Vitis acceleration flow for firmware development.

Architecture of the Kria App:



B. Preparing the KV260 and Raspberry Pi Camera v2 Setup

1. Necessary Hardware -

- KV260 Board , power adapter , LAN Cable, 16+ GB of SD Card – [Purchase Link from AMD](#)
- Raspberry Pi Camera v2 – [Purchase link from Raspberry pi-org](#)

2. Preparing the KV260 Board with Yocto EDF flow image

- Download and burn this wic image with BalenaEtcher - [AMD Download \[2025.2 image\]](#)
- Update the bootfirmware - [AMD Download - K26-BootFW-v1.06.bin](#) Update or later. Follow boot image recovery tool - [Kria+SOMs+Starter+Kits#Boot-Image-Recovery-Tool](#)
- Now plug the SD card to KV260 , connect Power adapter, Raspberry Piv2 MIPI camera and boot the board.

3. Connect the Pmod-UART and KV260 UART on the board with PC.

- The J4 (JTAG/UART) connection is primarily required to determine the IP address assigned to the KV260 once the boot process is complete. After obtaining the IP address, it can be used to access the KV260 via SSH.
- You can use MobaXterm/Teraterm in **Windows** or Gtterm in **Linux**.
- Connect the baudrate of **115200** for both of the UARTs

4. KV260 and Raspberry Pi v2 sensors and connection:



Picture – KV260 with Raspberry Pi Camera V2 (picture ref – misoji engineer)

5. Now boot the Kv260 board.

- After booting , you will start getting boot log from Yocto EDF into your PC/computer.
- After boot completes, you can enter the
 - Username: amd-edf

- Password: Will be prompted to change on first login
- c. **[Optional]** You can also identify and node down the KV260's IP address by running the **ifconfig** command at the terminal of KV260. Once you have identified the IP address, you can also connect KV260 via SSH either by MobaXterm or terminal or other application according to the OS.

C. Clone Github Repository

Clone this github repository at the KV260-Yocto-AMD-EDF image by running git clone command:

```
git clone https://github.com/LogicTronixInc/Kria-Yocto-Vitis-Vision-Library.git
```



D. Setup OpenCV, Media-ctl , V4L2 and utilities

The AMD-EDF image needs to setup for OpenCV and other utilities.

One can use the dnf approach to install those:

- `sudo dnf repolist`
- `sudo dnf install opencv`
 - `sudo dnf install -y opencv-dev`
- `sudo dnf install -y v4l-utils`
- `sudo dnf install gstreamer1.0-plugins-good`
- `sudo dnf install gstreamer1.0-plugins-base`
- `sudo dnf install xrt-dev`
- `sudo dnf dnf install weston`
- `sudo dnf install packagegroup-core-buildessential`



E. Run the Application

Follow this Readme to run the application on KV260

Here are also the steps brief:

1. Goto the /firmware directory of the cloned repo and copy the firmware into “/lib/firmware/Xilinx” at KV260 image.
2. Unload the default firmware and load this Colordetect firmware.
3. Now goto “HostApp/MIPI-ISP-IP-Config” and run this command `bash config-imx219-HD.sh`. This command will setup the MIPI pipeline for proper colorformat and resolution. Now run `bash capture\_jpg.sh`, this will capture new frame and save into “hoge.jpeg”. If you are able to visualize then check the jpeg and confirm pipeline is working.

4. To run the OpenCV Host APP

Goto `HostApp/OpenCV-CPP-PS-ColorDetect` and run this command:

```
./colordetect_ocv
```

This command will save the image from at where the colordetect algorithm is ran and output is generated, bounding box generated.

5. To run the XRT ColorDetect Host APP

Goto `HostApp/xrt-ColorDetect-app` and run following command:

```
./colordetect_app --xclbin binary_container_1_sept212.xclbin --source  
tpg --device /dev/video0 --frames 100 --save-frames 1 --out-dir  
./frames
```

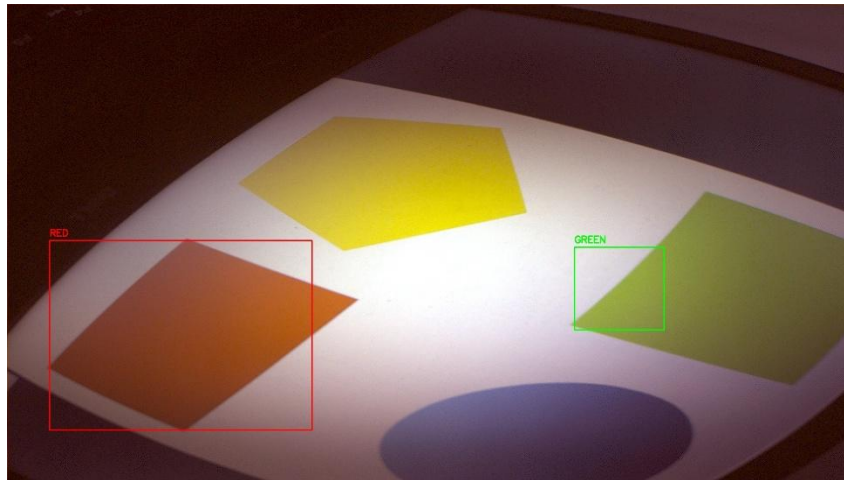
After running above command you will start seeing log like this:

```
[imx219] 1920x1080 -> kernel 1920x1080 (zero-copy path)  
[save] imx219_frame_0: 24 box(es)  
[timing] colordetect_accel (PL, imx219): n=100 avg=60.311ms  
min=60.0196ms max=62.0013ms throughput=16.5807fps
```

F. Compare the Accelerated ColorDetect and OpenCV ColorDetect at PS of MPSoC

While running both of the App, XRT Host APP and OpenCV C++ app, it will print the time taken by the core ColorDetect algorithm run. One can run the app multiple iteration and can get the respective performance metrics.

Here is the sample output:



Congratulations for successful run and completion of this Kria-App.

For any queries, please raise the GitHub issue or mail us at info@logictronix.com.

Revision History

S.N	Revision Number	Date of Revision	Revision Details	Author
1.	v1.0	June 21, 2026	Initial Revision	Krishna Gaihre

